



**Faculdade de Tecnologia SENAI “Antônio Adolpho Lobbe”
Unidade Vinculada “Engº Octávio Marcondes Ferraz” - Ribeirão Preto**

CST em Análise e Desenvolvimento de Sistemas

**Desenvolvimento Back-End
Prof. MSc. Gustavo M. N. Avellar**

Entrega 01 - Lista de Exercícios no estilo Leet Code sobre Lógica de Programação em Java

Instruções de entrega: A lista de exercícios é individual e deverá ser desenvolvida em Java do tipo “console” e organizada para entrega em um único repositório no GitHub. O repositório deverá conter todos os exercícios propostos. Cada exercício deverá ser armazenado em uma pasta própria, identificada pelo número e/ou nome do exercício. Você enviará o link para o repositório no Classroom.

Metodologia de resolução dos exercícios: Para cada dificuldade encontrada, siga esta ordem:

- 1. Tente sozinho por até 30 minutos:** Analise o problema, defina entradas, processamento e saídas e tente implementar a solução.
- 2. Consulte a documentação do Java (quantidade de tempo livre):** Pesquise sintaxe, classes, métodos e estruturas.
- 3. Faça uma pesquisa textual (quantidade de tempo livre):** Consulte artigos, fóruns e tutoriais, sem copiar soluções completas.
- 4. Consulte vídeos (quantidade de tempo livre):** Busque explicações quando os materiais anteriores não forem suficientes.
- 5. Use Inteligência Artificial:** Peça explicações, pistas, correções e sugestões. Não solicite a solução completa do exercício.

EXERCÍCIOS

1. Situação acadêmica do estudante

Desenvolva um programa que leia o nome de um estudante, três notas entre 0 e 10 e seu percentual de frequência, entre 0 e 100. O programa deverá calcular a média aritmética das notas e determinar a situação acadêmica do estudante de acordo com as seguintes regras:

- frequência inferior a 75%: reprovado por frequência;
- frequência igual ou superior a 75% e média igual ou superior a 7: aprovado;
- frequência igual ou superior a 75% e média entre 5 e 6,99: recuperação;
- frequência igual ou superior a 75% e média inferior a 5: reprovado por nota.

O cálculo da média e a determinação da situação deverão ser realizados por métodos separados. Ao final, o programa deverá apresentar o nome, a média com duas casas decimais, a frequência e a situação do estudante.

2. Validação e análise de uma data

Desenvolva um programa que leia três números inteiros correspondentes ao dia, ao mês e ao ano de uma data. O programa deverá verificar se a data informada é válida, considerando:

- a quantidade de dias de cada mês;
- anos bissextos;
- valores inválidos para dia, mês ou ano.

Caso a data seja válida, o programa deverá informar também qual trimestre do ano ela pertence e qual é sua posição no ano, considerando 1º de janeiro como o dia 1. Utilize métodos separados para verificar se um ano é bissexto, obter a quantidade de dias de determinado mês, validar a data e calcular sua posição no ano.

3. Calculadora interativa

Desenvolva uma calculadora de console que apresenta repetidamente o seguinte menu:

1. soma;
2. subtração;
3. multiplicação;
4. divisão;
5. potenciação;
6. resto da divisão;
7. encerrar.

Após a seleção de uma operação, o programa deverá solicitar os valores necessários, executar o cálculo e apresentar o resultado. Cada operação deverá ser implementada em um método próprio. Divisões por zero e opções inexistentes deverão ser tratadas sem encerrar inesperadamente o programa.

4. Analisador de texto

Desenvolva um programa que leia uma frase completa e produza uma análise de seu conteúdo. O programa deverá informar:

- quantidade de caracteres;
- quantidade de letras;
- quantidade de vogais;
- quantidade de consoantes;
- quantidade de algarismos;
- quantidade de espaços;
- quantidade de outros caracteres;
- quantidade de palavras;
- maior palavra encontrada;
- frequência de uma letra escolhida pelo usuário.

O programa também deverá verificar se a frase é um palíndromo, desconsiderando espaços, pontuação e diferenças entre letras maiúsculas e minúsculas. Organize cada operação principal em um método separado e percorra a string utilizando estruturas de repetição.

5. Estatísticas de uma sequência numérica

Desenvolva um programa que leia uma quantidade indeterminada de números inteiros. A leitura deverá continuar enquanto o usuário não informar o valor zero. Ao final, o programa deverá informar:

- quantidade de números digitados;
- soma de todos os valores e média geral;
- maior e menor valor;
- quantidade de valores positivos e negativos;
- quantidade de valores pares e ímpares;
- quantidade de múltiplos de três.

O programa também deverá tratar a situação em que o primeiro valor informado seja zero, evitando cálculos inválidos. Crie métodos auxiliares para as classificações numéricas utilizadas.

6. Ranking de tempos de uma competição

Desenvolva um programa para registrar os resultados de uma competição. Inicialmente, o usuário deverá informar a quantidade de participantes. Em seguida, deverão ser armazenados em vetores paralelos:

- o nome de cada participante;
- o tempo, em segundos, utilizado para concluir a prova.

O programa deverá:

- calcular o tempo médio;
- identificar o participante mais rápido e o mais lento;
- informar quantos participantes ficaram abaixo do tempo médio;
- ordenar os participantes do menor para o maior tempo;
- apresentar na tela o ranking completo com os nomes e tempo de cada participante;
- calcular a mediana e o desvio padrão dos tempos.

A ordenação deve ser implementada pelo próprio estudante, sem utilizar métodos prontos de ordenação. As operações principais deverão ser divididas em métodos.

7. Pares que atingem uma soma desejada

Desenvolva um programa que leia um vetor de números inteiros e um valor alvo. O programa deverá localizar todos os pares de posições diferentes cujos valores somados sejam iguais ao alvo. Para cada par encontrado, deverão ser apresentados:

- os dois valores;
- os índices em que eles aparecem;
- a soma obtida.

Uma mesma posição do vetor não poderá ser utilizada duas vezes no mesmo par e os pares não poderão ser repetidos em ordem inversa. Por exemplo, um par formado pelos índices 2 e 5 não deverá ser apresentado novamente como 5 e 2.

Ao final, o programa deverá informar a quantidade total de pares encontrados. Caso nenhum par exista, deverá apresentar uma mensagem apropriada. A busca deverá ser implementada em um método que percorra o vetor.

8. Gerenciador de lista de compras

Desenvolva um programa para gerenciar uma lista de compras utilizando estruturas `ArrayList`. Para cada produto, deverão ser armazenados seu nome, sua quantidade e seu preço unitário. Como ainda não serão utilizadas classes próprias, essas informações deverão ser mantidas em listas paralelas. O programa deverá apresentar continuamente as seguintes opções:

1. adicionar um produto;

2. alterar a quantidade de um produto;
3. alterar o preço de um produto;
4. remover um produto;
5. pesquisar produtos pelo nome ou por parte dele;
6. listar todos os produtos;
7. calcular o valor total da compra;
8. identificar o produto com maior subtotal;
9. encerrar.

O subtotal de um produto corresponde à multiplicação de sua quantidade pelo preço unitário. O programa deverá impedir quantidades negativas, preços inválidos e inconsistências entre as posições das listas. Ao adicionar um produto que já exista, o programa deverá oferecer a possibilidade de atualizar sua quantidade, em vez de criar uma duplicação.

9. Sistema de reservas de assentos

Desenvolva um programa que represente os assentos de uma sala de cinema por meio de uma matriz. Cada posição deverá indicar se o assento está livre ou ocupado.

O programa deverá apresentar um menu com as seguintes operações:

1. exibir o mapa de assentos;
2. reservar um assento;
3. cancelar uma reserva;
4. informar a quantidade e o percentual de assentos ocupados;
5. identificar a fileira com maior ocupação;
6. procurar um conjunto de X assentos consecutivos;
7. encerrar.

Na busca por assentos consecutivos, o usuário deverá informar a quantidade desejada e o programa deverá localizar o primeiro conjunto disponível em uma mesma fileira. Caso exista, deverão ser informadas a fileira e as posições encontradas. O programa deverá validar os limites da matriz, impedir a reserva de assentos já ocupados e impedir o cancelamento de assentos livres. Cada operação deverá ser implementada em um método separado.

10. Caça palavras multidirecional

Desenvolva um programa que represente um caça palavras por meio de uma matriz de caracteres. O usuário deverá informar as dimensões da matriz, as letras de cada posição e uma lista de palavras que deverão ser procuradas.

Cada palavra poderá aparecer em qualquer uma das oito direções:

- horizontal da esquerda para a direita;
- horizontal da direita para a esquerda;
- vertical de cima para baixo;
- vertical de baixo para cima;
- diagonal superior esquerda para inferior direita;
- diagonal inferior direita para superior esquerda;
- diagonal superior direita para inferior esquerda;
- diagonal inferior esquerda para superior direita.

Para cada palavra, o programa deverá informar se ela foi encontrada. Em caso positivo, deverá apresentar a posição inicial, a posição final e a direção em que foi localizada. As buscas não poderão ultrapassar os limites da matriz nem continuar do lado oposto ao atingir uma borda. Palavras poderão compartilhar letras. A verificação de uma palavra em determinada posição e direção deverá ser realizada por um método específico, reutilizado durante toda a busca.